

Astrazione del controllo

→ Verificare cosa è necessario descrivere, mantenere pubblico.

E cosa possiamo nascondere $\Rightarrow$ così possiamo focalizzare la programmazione sulle questioni importanti?

ottimizzare implementazione
ridurre errori
migliorare leggibilità ...

$\Rightarrow$ introduce legami che vanno gestiti.

no astrazione del controllo no **comandi**

Nome no riferisce ad un insieme di comandi

Nome $\rightarrow$ [codice che esegue sempre
nello stesso nodo

Parametri: no Permettono di usare un pezzo di codice su valori (input) differenti.
 $\Rightarrow$ eseguono computazioni simili richiamate usando lo stesso nome

tipo + risultato $\equiv$ interfaccia

La parte nascosta dell'astrazione $\rightarrow$ corpo della procedura $\equiv$ codice eseguito

tipo valore di ritorno nome
double P(int x) { $\rightarrow$ interfaccia
 double z;
 /* corpo */
 return expr; $\rightarrow$ corpo
}

$\rightarrow$ lista (nomi e tipo) di parametri:

Per definire procedure il PL deve avere strumenti sintetici:

$\rightarrow$ per specificare (definire) P (interfaccia) } definizione completa di P $\rightarrow$ dichiarazioni
 $\rightarrow$ scrivere P
 $\rightarrow$ visualizzare P (chiamare l'esecuzione del corpo di P) $\rightarrow$ comandi

specifica

```
int foo (int n, int a) { blocco
```

nome
int tmp = a;

definizione
if (tmp == 0) then return n;
else return n+1;

dichiarazione

ambiente locale:

- dichiarazione interne
- parametri formali (parametri)

```
int x = foo (3, 0); chiamata comando
```

nome

parametri attuali → valori con cui si vogliono
inizializzare i parametri (bind)
della procedura

contesto
del chiamante

```
int foo (int n) {
```

```
int tmp = a;
```

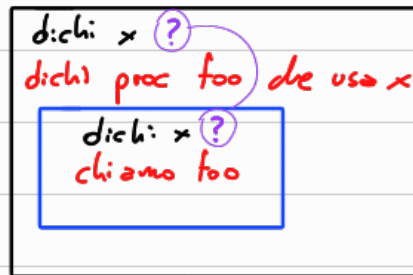
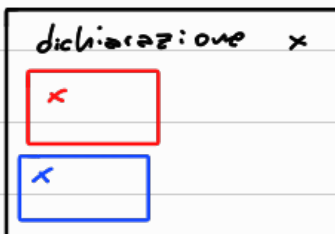
```
}
```

identificatore non più identificato libero
locale

Dove trovare significato (legame)
nel contesto

PROBLEMA: Nel contesto (chiamante) potrebbero esserci più definizioni di a, quale usiamo?

Una dichiarazione locale ad un blocco è visibile nel blocco e in tutti i blocchi in questo annidati: a meno di ridefinizioni dello stesso identificatore



Regola di visibilità non è sufficiente a risolvere i riferimenti quando usiamo le procedure (se la procedura HA ambiente non locale)

PROBLEMA: capire quale ambiente va usato come riferimento per l'ambiente non locale della procedura

→ Va definita la regola di scoping

- statico l'ambiente è quello della definizione
- dinamico l'ambiente è quello della chiamata

[0:42:00]

Ambiente locale $\rightarrow$ dich. local:
 $\rightarrow$ comunit.

→ regole di passaggio di parametri:

Ambiente di riferimento di una procedura

- Regole di visibilità (di standard di annidamento)
- Regole di scoping (dove risolvere ambienti non locali di codice eseguito in contesti diversi da quelli di definizione)
- Regole per passaggio di parametri
- Regole specifiche del PL

regola di binding

inf f (g funzione...)

funzione come parametro di un'altra funzione

- def g $\rightarrow$ scoping static

- passaggio da f ad g

- esecuzione di g dentro f

[0:48:54]

```
{ int x = 10;
void foo () {
    x++;
}
```

```
void tie () {
```

```
int x = 0;
```

foo().

3

 $f_e()$

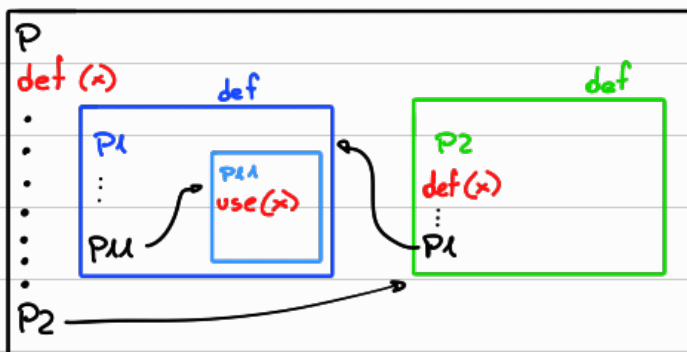
3

ambiente non locale (x non è parametro formale e non è dichiarata nel blocco)

scoping & dynamic

scoping
statico

SCOPING STATIC (C, Java, py, JS)



Nella sequenza di annidamento statico (delle definizioni):

P11 NON VEDE P2

↳ uso di x in $PL1$ si riferisce alla def di x in PL

$P \left[\begin{array}{l} P_1 \\ \vdots \\ P_2 \end{array} \right]$
 Applichiamo la regola di visibilità usando come relazione di annidamento quello di definizione

chiamate $P \rightarrow P_2 \rightarrow P_1 \rightarrow P_{11}$

definizione $P \rightarrow P_2 \rightarrow P_1 \rightarrow P_{11}$ *scoping statico* la risoluzione dei riferimenti avviene a tempo di compilazione

```

{ int x=0
void pippo (int n){
  x = n+1;
}
  
```

x non locale
 n locale

$pippo(3); \rightarrow$ modifico x (globale) $\rightarrow 4$

$write(x); \rightarrow 4$

```

{ int x=0;
  pippo(4);
  write(x);
}
  
```

$\rightarrow$ nello scoping statico è SEMPRE quella globale

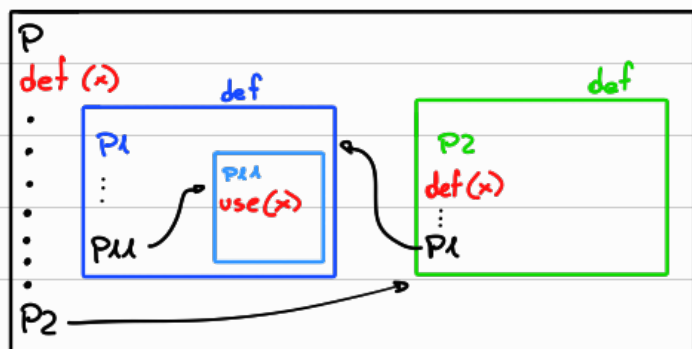
$write(x); \rightarrow 5$

$\Rightarrow$ scoping statico garantisce l'indipendenza (dal riferimento) dalla posizione della chiamata

$\left[\begin{array}{l} - \text{def} \\ \downarrow \text{visibilità} \end{array} \right]$

$\rightarrow$ rende il codice più comprensibile
 $\rightarrow$ più difficile da implementare

SCOPING DINAMICO



Chiamate: $P \rightarrow P_2 \rightarrow P_1 \rightarrow P_{11}$

Applica la regola di visibilità usando come "relazione" di annidamento quella della sequenza di chiamate

$\Rightarrow$ Il riferimento (legame) corretto dipende dalla posizione della chiamata

$\Rightarrow$ chiamate diverse fanno riferimento a legami diversi